Pembelajaran Mendalam
IF25 - 40401

Mgg1 : Pengenalan *Deep Learning*



Tim Penyusun: IEW, RIK, MCT, MCU

Semester Ganjil T.A. 2026-2027
Teknik Informatika, Fakultas Teknologi Industri
Institut Teknologi Sumatera

Pokok Bahasan

- Kontrak Perkuliahan
- Pengenalan *Deep Learning* (DL)
- *DL-Environment*
- *Activation Function, Loss Function, Optimizer*
- *Weight Initialization*
- *Performance Metrics*
- *Multi Layer Perceptron (Review)*

Dosen Pengampu Mata Kuliah



Imam Ekowicaksono, S.Si., M.Si. (IEW)
imam.wicaksono@if.itera.ac.id



Martin C.T. Manullang, P.hD. (MCT)
martin.manullang@if.itera.ac.id



Rahman Indra Kesuma, S.Kom., M.Cs. (RIK)
rahman.indra@if.itera.ac.id



Meida Cahyo Untoro, S.Kom., M.Kom. (MCU)
cahyo.untoro@if.itera.ac.id

Identitas Mata Kuliah

- Bobot/Kredit: 3 SKS
- Mata Kuliah Prasyarat:
  - Inteligensi Buatan;
- Peraturan Perkuliahan:
  - Jadwal: Rabu 07.30-10.10 (RA dan RB);
  - Perkuliahan Daring: seluruh mahasiswa wajib mengaktifkan kamera, dalam posisi duduk, dan berpakaian rapih;
  - Perkuliahan Luring: toleransi keterlambatan 15 menit. Setelahnya, mahasiswa diizinkan mengikuti perkuliahan tetapi tidak mendapatkan akses presensi;
- Capaian Pembelajaran MK
  - CPMK-AIDE-01 : Mahasiswa mampu membangun basis keputusan aplikatif dengan mengoptimalkan kinerja komputasi melalui pemrosesan simultan, serta menerapkan prinsip etika komputasi.
  - CPMK-AIDE-02 : Mahasiswa mampu merancang dan mengimplementasikan pemodelan kecerdasan buatan lanjut untuk menganalisis aneka data bervolume besar melalui pengelolaan infrastruktur penyimpanan data terstruktur.

Identitas Mata Kuliah

- E-Learning : *https://kuliah2.itera.ac.id*
  - Course Name : “IF25-40401: Pembelajaran Mendalam (Deep Learning)”
  - Enrolment Key untuk Kelas RA : DeepLearningRA#2026
  - Enrolment Key untuk Kelas RB : DeepLearningRB#2026

Komponen Penilaian Mata Kuliah

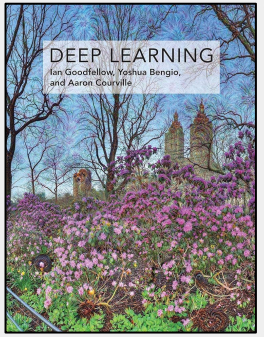
- Quiz (Bobot 10%)
  - Quiz Pertama akan Dilaksanakan Pra UTS
  - Quiz Kedua akan Dilaksanakan Pra UAS
- Ujian Tengah Semester (Bobot 10%)
- Tugas Studio Proyek (Bobot 30%)
  - Studio Pertama terkait Penerapan ANN pada CIFAR-10
  - Studio Kedua terkait Penerapan CNN pada ImageNet
  - Studio Ketiga terkait Penerapan Transformer pada Domain NLP
- Tugas Besar sebagai UAS (Bobot 50%)

Pokok Bahasan Perkuliahan

- Pengenalan dan Traning Deep Learning Model (Mgg1-2)
- Studio Proyek 1: Fine-Tuning ANN from Scratch pada CIFAR-10 (Mgg3)
- Deep Computer Vision (Mgg4-5)
- Studio Proyek 2: Fine-Tuning CNN Pretrained Pada ImageNet (Mgg6)
- Deep Sequence Modelling & Attention Introduction (Mgg7)
- Arsitektur Transformer pada Domain NLP (Mgg9)
- Studio Proyek 3: Fine-Tuning Transformer Pretrained Untuk NLP (Mgg10)
- Advanced Deep Learning (Mgg 11-12)
  - Generative Models — Autoencoder & GAN
  - Frontier — RL, GNN & Mamba
- Coaching Clinic - Pendampingan Dan Finalisasi Tugas Besar (Mgg 13-14)

Referensi / Pustaka

- I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P., 2002. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), pp.2278-2324.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in neural information processing systems, 30.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S. and Uszkoreit, J., 2020. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. and Riedmiller, M., 2013. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y., 2020. Generative adversarial networks. Communications of the ACM, 63(11), pp.139-144.
- F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner and G. Monfardini, "The Graph Neural Network Model," in IEEE Transactions on Neural Networks, vol. 20, no. 1, pp. 61-80, Jan. 2009, doi: 10.1109/TNN.2008.2005605.
- Gu, A. and Dao, T., 2023. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752.





Pengenalan *Deep Learning*

- *DL-Environment*
- *Activation Function, Loss Function, Optimizer*
- *Weight Initialization*
- *Performance Metrics*
- *Multi Layer Perceptron (Review)*



DL-Environment dalam IF25-40401



python™

PyTorch

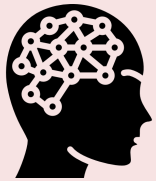
ANACONDA®

jupyter

Apa itu *Deep Learning* ??

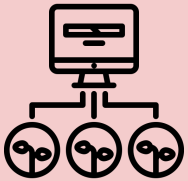
Artificial Intelligence

Beragam teknik untuk membuat komputer meniru tingkah laku dan kecerdasan manusia



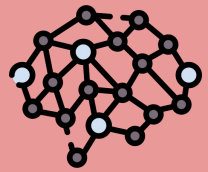
Machine Learning

Kemampuan untuk belajar dari data tanpa diprogram secara eksplisit



Deep Learning

Ekstrak pola dari data menggunakan neural network dengan jaringan yang lebih dalam

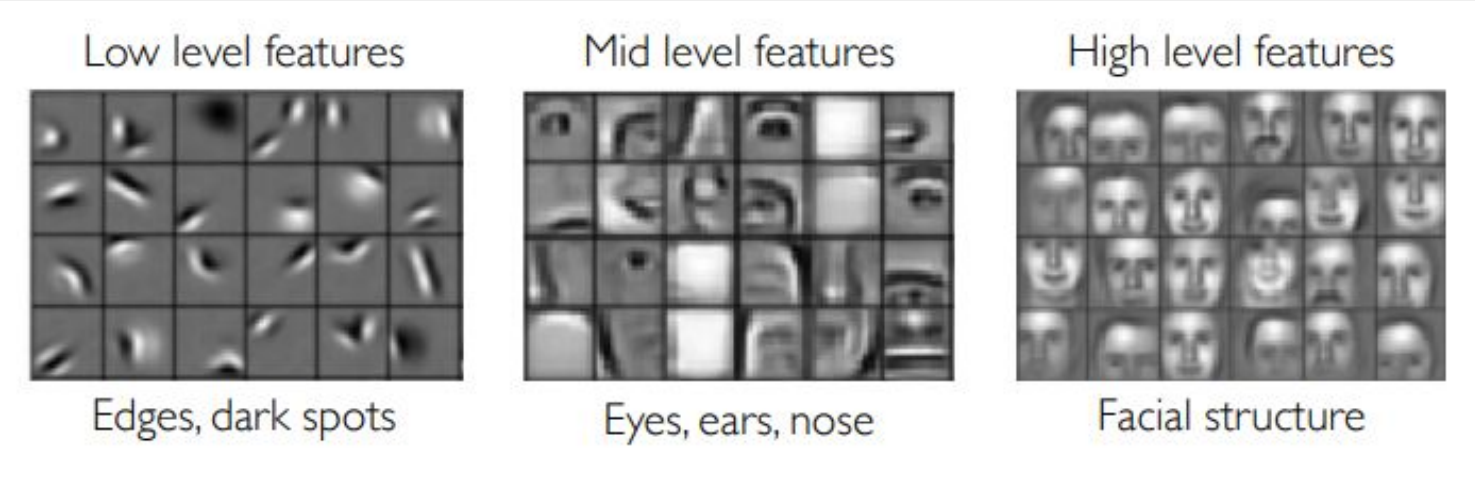


Aktivitas untuk mengajarkan komputer **bagaimana cara mempelajari suatu pekerjaan (*task*)** secara langsung dari **data mentah (raw data)**

Mengapa Dibutuhkan *Deep Learning* ??

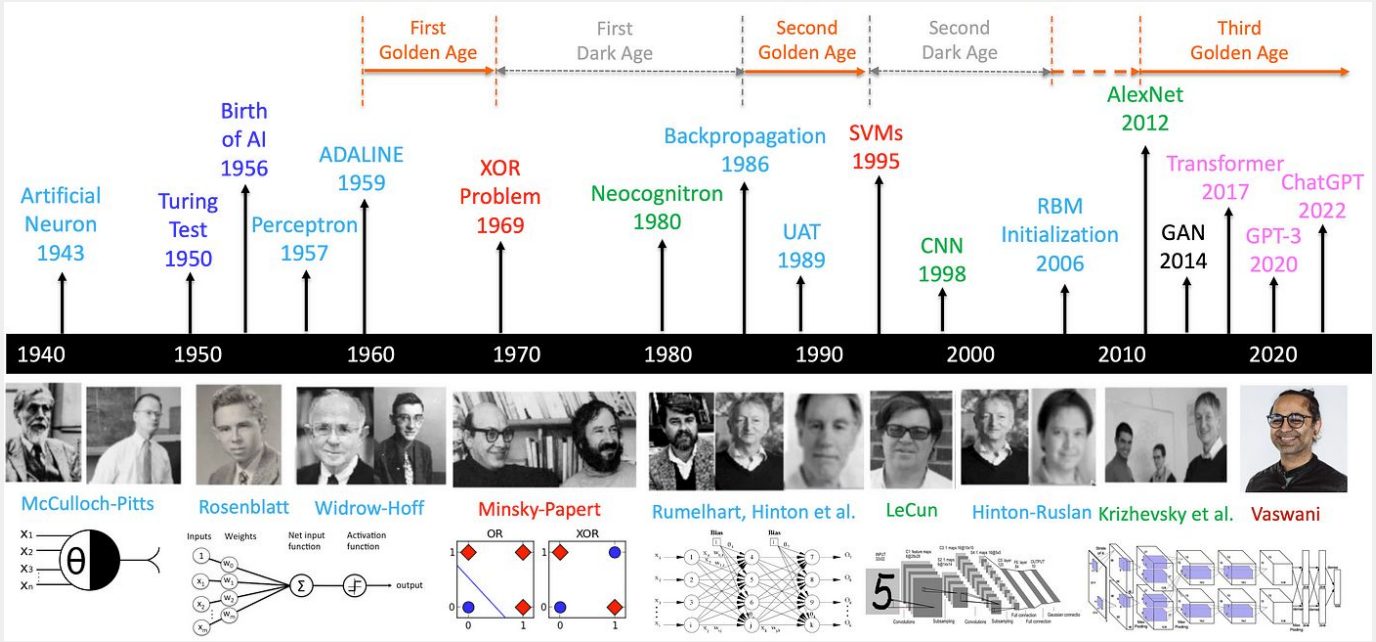
Fitur yang dihasilkan dari proses konvensional (***hand engineered features***) membutuhkan waktu lebih, tidak cukup tangguh, dan tingkat skalabilitas yang rendah untuk penerapan di kehidupan nyata.

Dapatkah mesin diajarkan untuk **memahami beragam fitur** secara langsung dari data?



Mengapa Bertahan Hingga Saat Ini ??

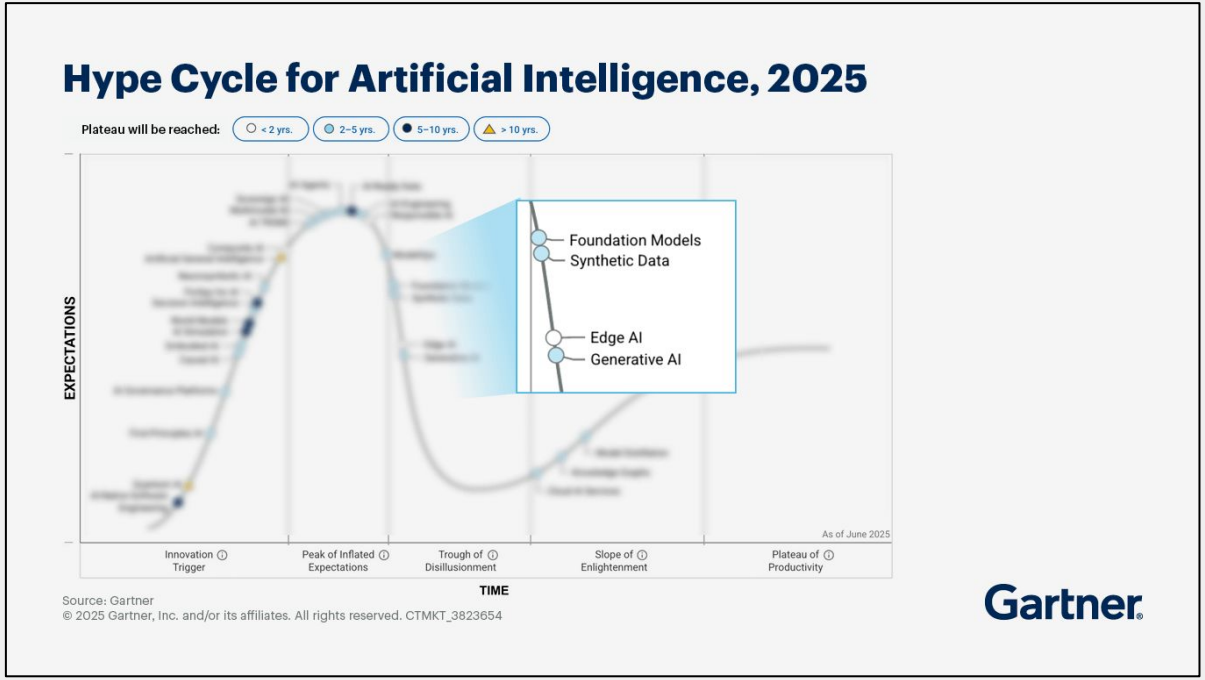
Neural network sudah ada sejak beberapa dekade yang lalu, mengapa masih dominan hingga saat ini?



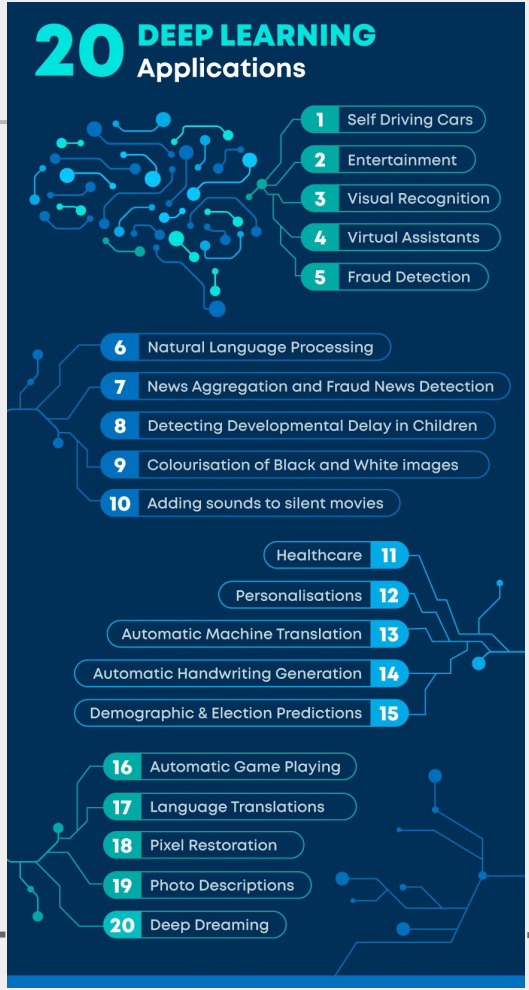
Link: https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

1. Big Data
   - Larger datasets
   - Easier collection and storage
2. Hardware
   - Graphics Processing Units (GPUs)
   - Massively Parallelizable
3. Software
   - Improved techniques
   - New models
   - Toolboxes

Trend Deep Learning dalam Riset AI

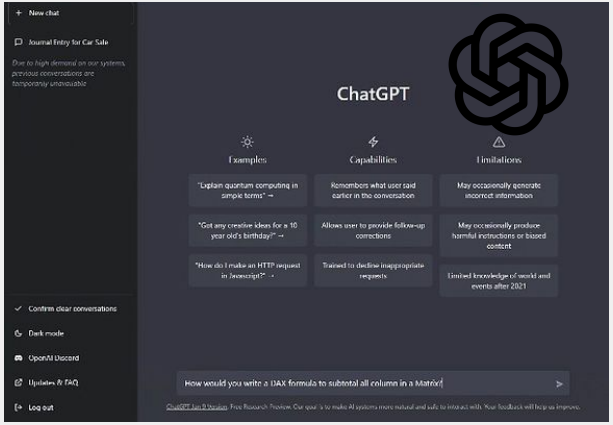


Link: https://www.gartner.com/en/articles/hype-cycle-for-artificial-intelligence



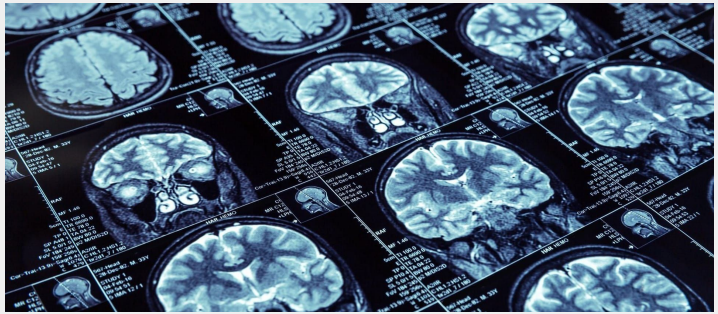
Link: https://www.mygreatlearning.com/blog/deep-learning-applications/

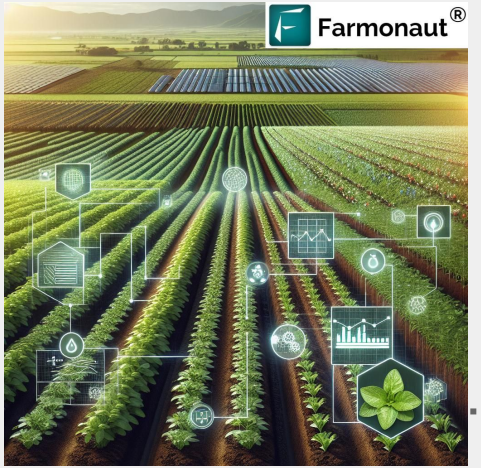
DL di Kehidupan Sehari-Hari ...







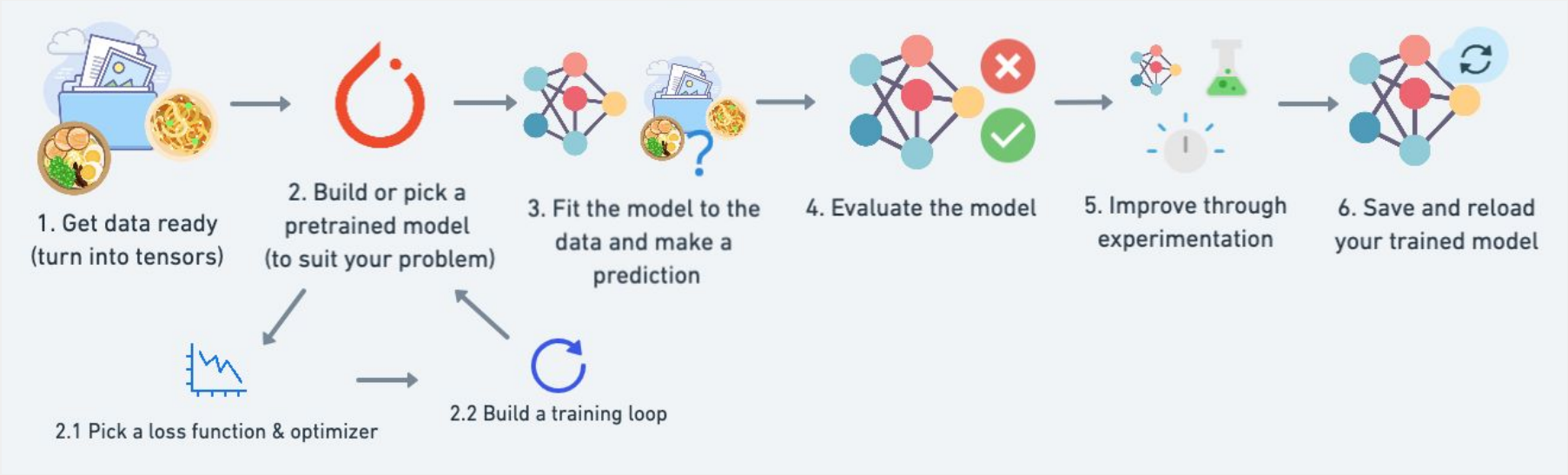






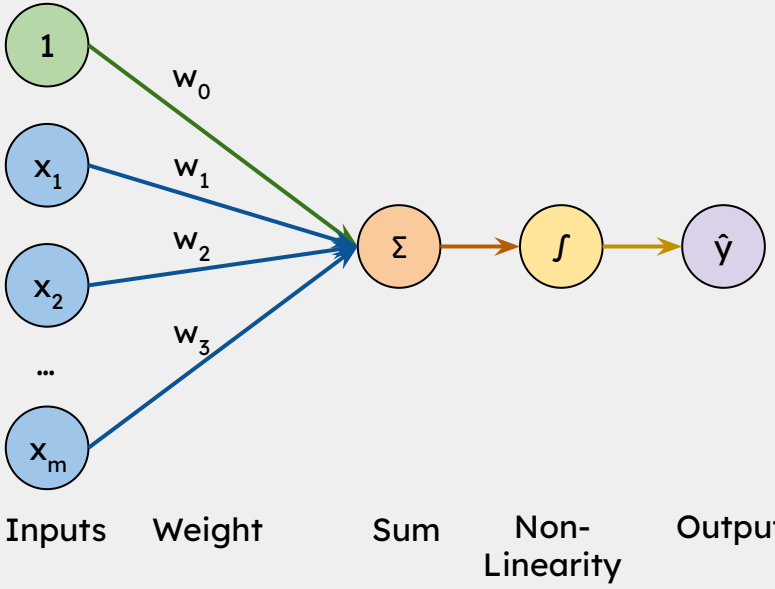
Tahapan Dasar dari Eksperimen DL

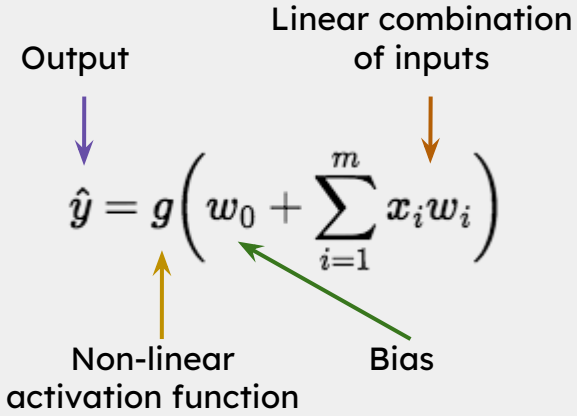
Tahapan dasar dari eksperimen deep learning sangat penting untuk dipahami karena proses pelatihan+validasi+pengujian dari model akan menggunakan tahapan dasar yang sama



Perceptron: Forward Propagation

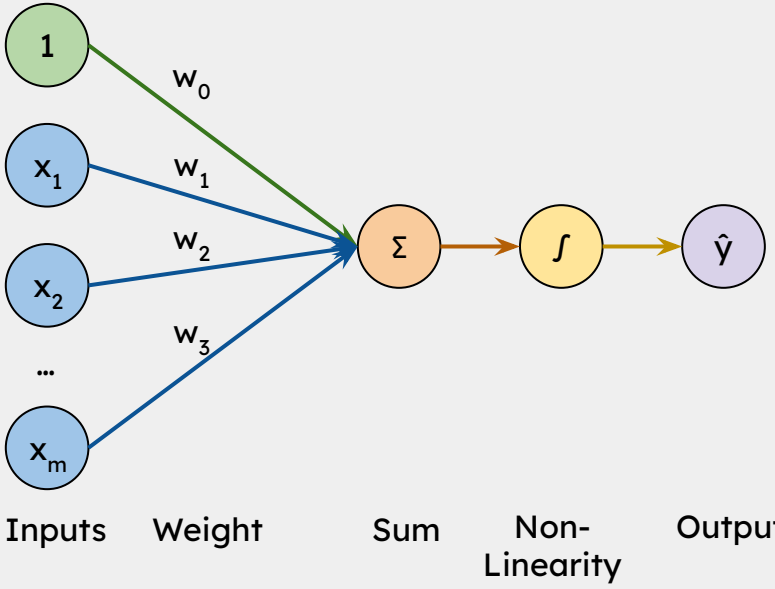
The structural building block of deep learning





Perceptron: Forward Propagation

The structural building block of deep learning



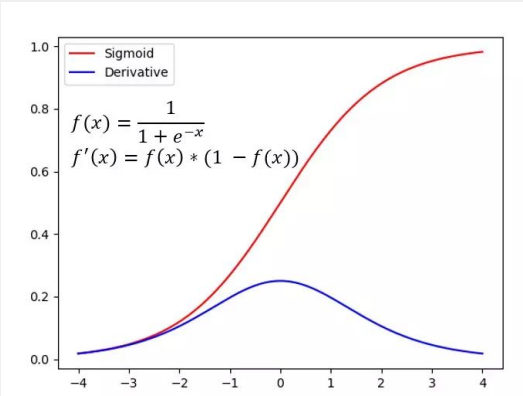
$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i w_i\right)$$

$$\hat{y} = g\left(w_0 + X^T W\right)$$

$$\text{dimana: } X = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ dan } W = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

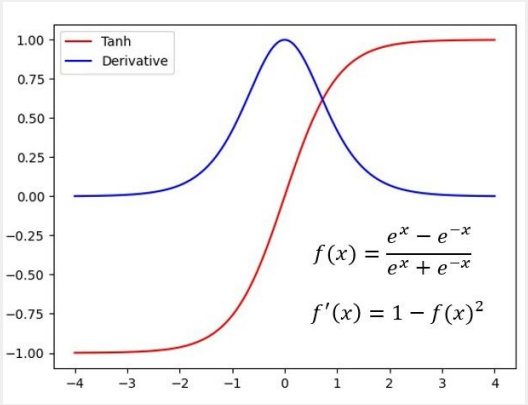
Fungsi Aktivasi

Berikut ini beberapa contoh fungsi aktivasi yang umum digunakan dalam arsitektur Deep Learning



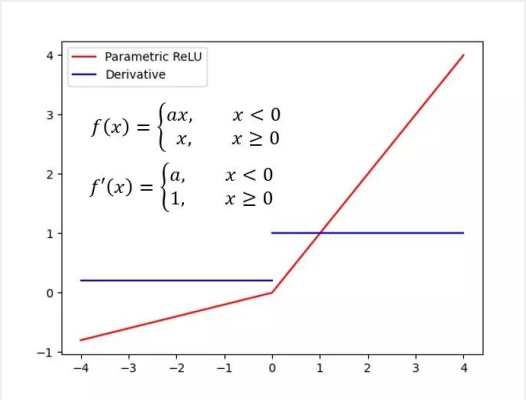
Fungsi Sigmoid

`torch.nn.Sigmoid(z)`



Fungsi Tanh

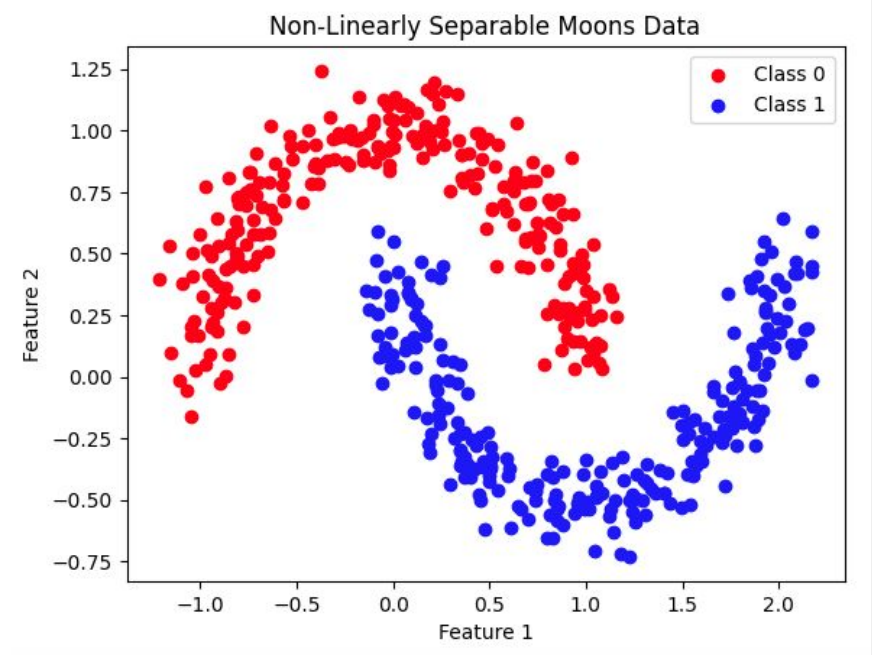
`torch.nn.Tanh(z)`



Fungsi Leaky ReLU

`torch.nn.LeakyReLU(z)`

Pentingnya Fungsi Aktivasi



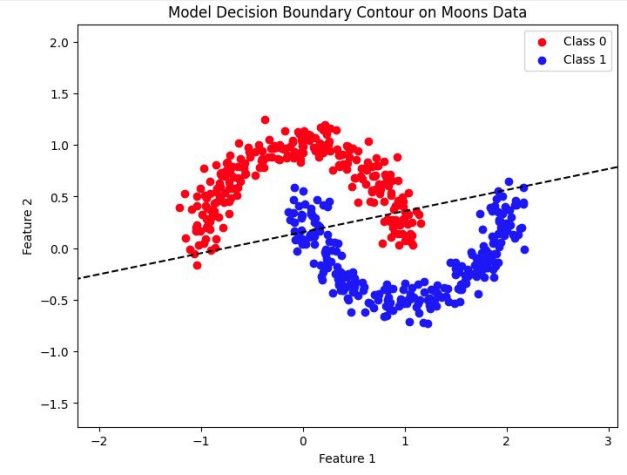
Tujuan dari fungsi aktivasi adalah untuk **memperkenalkan *non-linearities*** dalam jaringan

```
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt
import numpy as np
X, y = make_moons(n_samples=500,
                  noise=0.1,
                  random_state=42)
```

Bagimana jika ingin dibangun neural network untuk membedakan antara titik-titik merah dan biru?

Pentingnya Fungsi Aktivasi

Tujuan dari fungsi aktivasi adalah untuk **memperkenalkan *non-linearities*** dalam jaringan

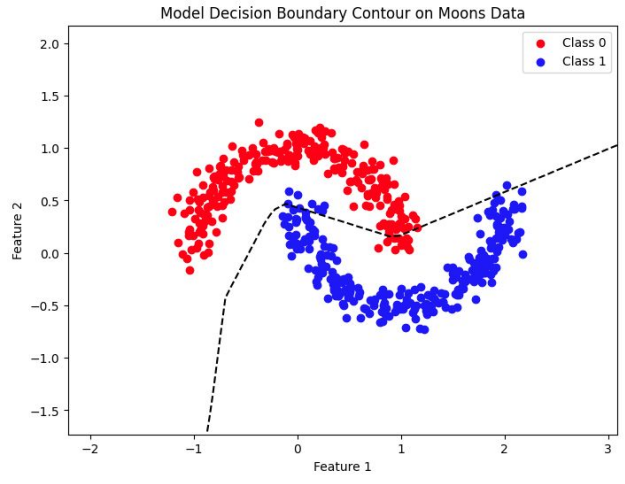



```
class ModelLinear(torch.nn.Module):
  def __init__(self, input_size, hidden_size, output_size):
    super(ModelLinear, self).__init__()
    self.fc1 = torch.nn.Linear(input_size, hidden_size)
    self.fc2 = torch.nn.Linear(hidden_size, hidden_size)
    self.fc3 = torch.nn.Linear(hidden_size, output_size)

  def forward(self, x):
    x = self.fc1(x)
    x = self.fc2(x)
    x = self.fc3(x)
    return x
```

Pentingnya Fungsi Aktivasi

Tujuan dari fungsi aktivasi adalah untuk **memperkenalkan *non-linearities*** dalam jaringan




```
class ModelLinearWithActFunc(torch.nn.Module):
  def __init__(self, input_size, hidden_size, output_size):
    super(ModelLinearWithRelu, self).__init__()
    self.fc1 = torch.nn.Linear(input_size, hidden_size)
    self.fc2 = torch.nn.Linear(hidden_size, hidden_size)
    self.fc3 = torch.nn.Linear(hidden_size, output_size)
    self.relu = torch.nn.ReLU()
    self.tanh = torch.nn.Tanh()

  def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    x = self.relu(x)
    x = self.fc3(x)
    x = self.tanh(x)
    return x
```



Loss Function & Optimizer

Dua komponen utama yang perlu ditentukan sebelum melakukan training loop:

1. Loss function, yang digunakan untuk mengukur biaya/hukuman yang harus diterima dari kesalahan prediksi.
2. Optimizer, yang digunakan untuk membuat pencarian learning rate set terbaik secara adaptif berdasarkan landscape dari ruang pencarian model terbaik.

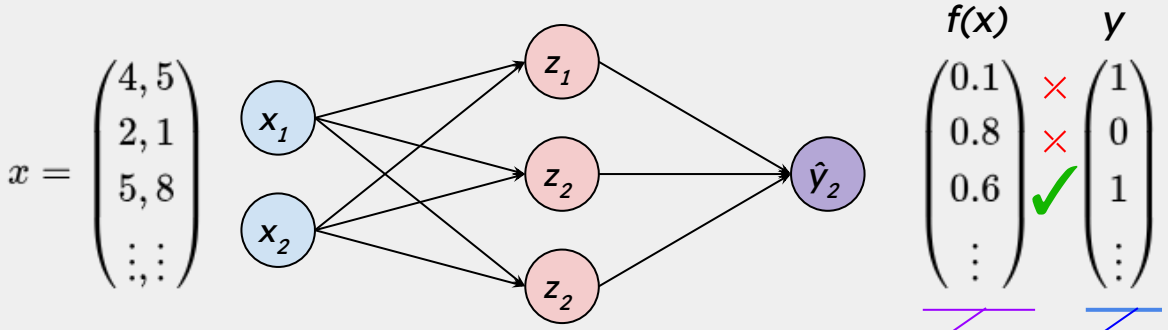
Binary Cross Entropy Loss
Categorical Cross Entropy Loss
Sparse Categorical Cross Entropy Loss
Hinge Loss

Mean Squared Error
Mean Absolute Error
Huber Loss

Stochastic Gradient Descent (SGD)
Adaptive Moment Estimation (Adam)
Adaptive Gradient Algorithm (Adagrad)
Adaptive Learning Rate Method (Adadelta)
Root Mean Square Propagation (RMSProp)

Loss Function

Empirical Loss mengukur total nilai loss dari keseluruhan data pada dataset yang diproses



Juga dikenal dengan:

- Objective function
- Cost function
- Empirical risk

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(f(x^{(i)}; W), y^{(i)} \right)$$

Loss Function: Binary Cross Entropy Loss

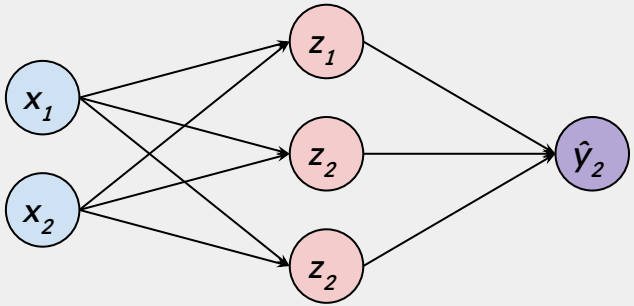
Entropy adalah alat ukur tingkat ***uncertainty***, ***disorder***, atau ***randomness*** dalam label atau kelas dari kumpulan data (dataset).

Cross Entropy atau juga dikenal logarithmic loss atau log loss, mengukur perbedaan antara distribusi probabilitas yang ditemukan dari model klasifikasi dan nilai yang diprediksi.

Cross Entropy Loss dapat digunakan untuk mengukur kualitas klasifikasi model yang memberikan luaran nilai probabilitas antara 0 dan 1.

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n} y^{(i)} log\big(f(x^{(i)};W)\big) + (1-y^{(i)}) log\big(1-f(x^{(i)};W)\big)$$

$$x = \begin{pmatrix} 4,5 \\ 2,1 \\ 5,8 \\ \vdots,\vdots \end{pmatrix}$$



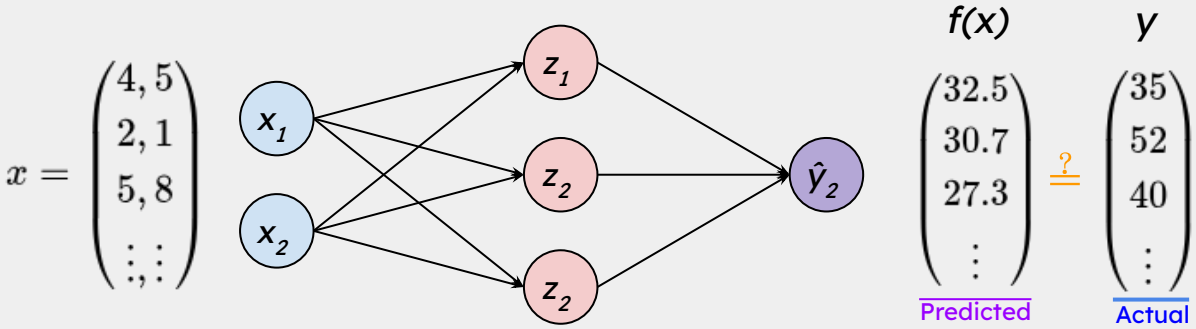
f(x) y

$$\begin{pmatrix} 0.1 \\ 0.8 \\ 0.6 \\ \vdots \end{pmatrix} \times \begin{pmatrix} 1 \\ 0 \\ 1 \\ \vdots \end{pmatrix}$$

Predicted Actual

Loss Function: Mean Squared Error Loss

Mean Squared Error (MSE) Loss dapat digunakan pada **model regresi** yang memberikan nilai luaran berupa angka dalam **bilangan real** (*continuous number*)

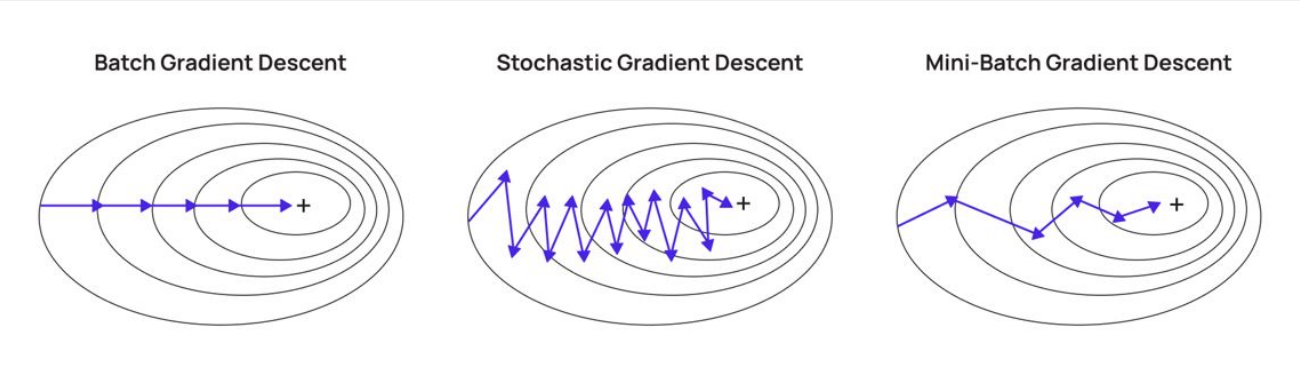


$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - f(x^{(i)}; W) \right)^2$$

Loss Function - In MiniBatch

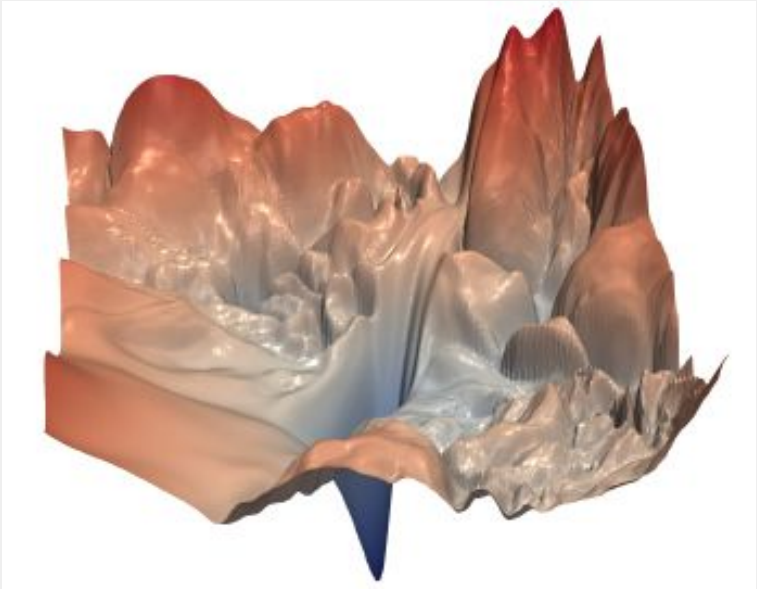
Untuk setiap mini-batch, nilai loss **dihitung untuk setiap data dalam batch** yang selanjutnya digunakan nilai **rata-rata**nya untuk nilai fungsi loss yang dipertimbangkan pada proses gradien.

Nilai loss yang dirata-ratakan pada mini-batch dapat membantu mengurangi noise ketika dibandingkan dengan pendekatan SGD.



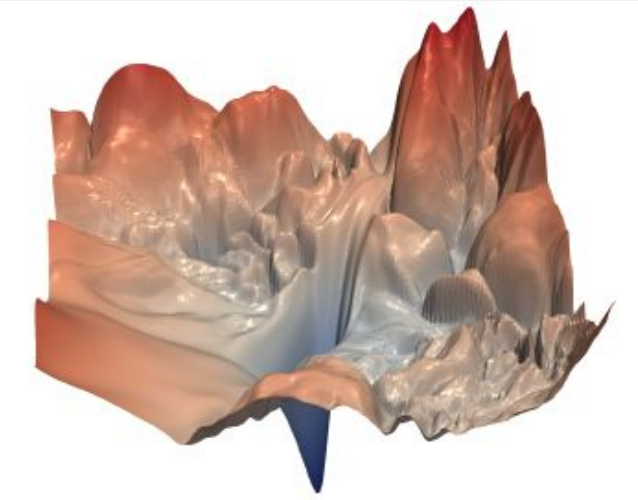
Melatih Model Tidaklah Mudah !!!

Mencari model terbaik untuk mendapatkan Loss pada posisi global optima adalah pekerjaan yang sulit



Melatih Model Tidaklah Mudah !!!

Loss fungsi menjadi sulit untuk dioptimalkan, tetapi mari ingat kembali proses optimasi melalui gradient descent

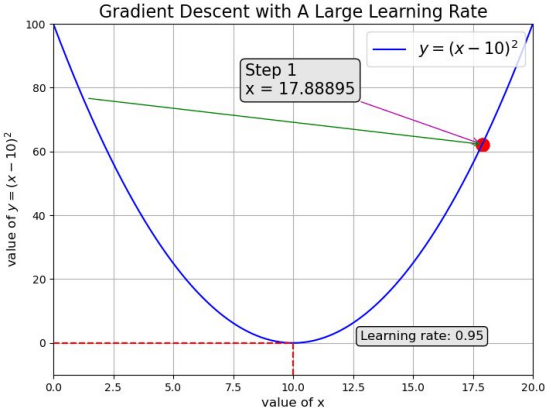


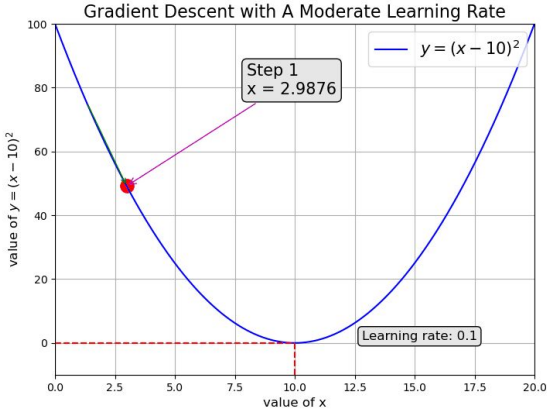
$$W = W - \eta \frac{dJ(W)}{dW}$$

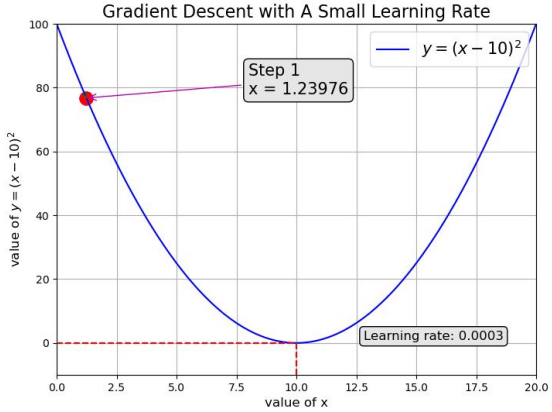
Bagaimana mengatur nilai learning rate?

- **Lr sangat kecil** membuat konvergensi sangat lambat dan cenderung terjebak pada local minima;
- **Lr sangat besar** bisa menjadi pencarian berlebihan (*overshoot*), tidak stabil, dan divergensi;
- **Lr ideal** bisa mengarahkan pada konversi model yang halus dan menghindari terjebak pada local minima;

Mencari Lr yang Ideal ???









Bagaimana cara untuk mendapatkan nilai *learning rate* yang ideal pada beragam *task*?

Mencari Lr yang Ideal ???

Umumnya terdapat dua cara yang umum digunakan:

Cara 1: Mencoba beragam nilai learning rate dalam rentang tertentu dan melihat apakah hasilnya sudah sesuai dengan harapan

Cara 2: Mencoba cara yang lebih pintar, dengan merancang learning rate yang dapat beradaptasi dengan landscape ruang pencarian loss terbaik

 Optimizer

Optimizer: Adaptive Learning Rates

Learning rate bukan lagi merupakan nilai yang bersifat tetap;

Dapat dibuat menjadi lebih besar atau lebih kecil berdasarkan pada:

- Seberapa besar nilai gradient;
- Seberapa cepat proses pembelajaran terjadi (perubahan nilai loss);
- Ukuran dari bobot dari suatu arsitektur;
- Dan lainnya .....

Penerapan Loss Function dan Optimizer

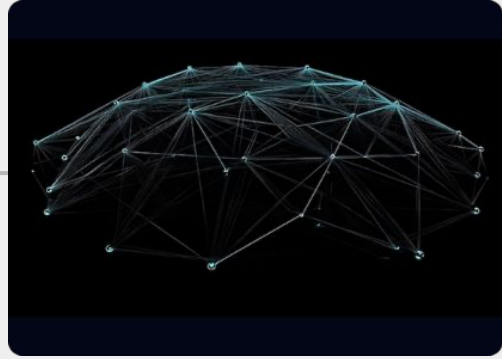
```
criterion = torch.nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

model.train()
loss_history = []
for epoch in range(epochs):
  running_loss = 0.0
  for inputs, labels in dataloader:
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.detach().cpu().numpy()
  print(f'Epoch {epoch+1}, Loss: {running_loss/len(dataloader)}')
  loss_history.append(running_loss/len(dataloader))
```

PyTorch

Weight Initialization



Mengapa Dibutuhkan & Keuntungannya?

- **Menentukan Titik Awal (Starting Point)**: Bobot awal menentukan posisi model dalam lanskap optimasi. Inisialisasi yang tepat mencegah model terjebak terlalu dini di local minima.
- **Memecah Simetri (Symmetry Breaking)**: Jika semua bobot diinisialisasi dengan angka yang sama (misal: nol), semua neuron akan memperbarui bobot secara identik. Pengacakan menyelesaikan masalah ini.
- **Konvergensi Lebih Cepat**: Distribusi bobot yang seimbang membuat proses Gradient Descent mencapai titik optimal dengan jumlah iterasi (epoch) yang jauh lebih sedikit.
- **Menjaga Stabilitas Sinyal**: Mencegah sinyal input memudar atau membesar secara eksponensial saat merambat melewati puluhan hidden layer.

Weight Initialization Cara Kerja & Metode Populer

Sistem menggunakan distribusi probabilitas (seperti Gaussian/Normal) untuk menghasilkan angka acak yang skalanya disesuaikan dengan dimensi layer.



Random Initialization

Memberikan nilai acak berskala kecil. Walaupun lebih baik daripada nilai nol, metode ini seringkali tidak stabil dan tidak direkomendasikan untuk arsitektur jaringan yang dalam (Deep Networks).



Xavier / Glorot

Menentukan varians bobot berdasarkan jumlah neuron input dan output. Sangat efektif untuk menjaga kestabilan ketika model menggunakan fungsi aktivasi Sigmoid atau Tanh.



He Initialization

Dikembangkan khusus untuk fungsi aktivasi ReLU (Rectified Linear Unit). Metode ini mengompensasi hilangnya separuh informasi sinyal akibat pemotongan nilai negatif oleh ReLU.

Weight Initialization

Mengapa Dibutuhkan & Keuntungannya?

Inisialisasi yang terlalu besar atau terlalu kecil akan menyebabkan kegagalan sistematis saat proses Backpropagation menghitung turunan (gradien).

Vanishing Gradient

Jika bobot awal terlalu kecil (< 1), gradien akan terus mengecil secara eksponensial di layer-layer awal. Jaringan akan berhenti belajar karena pembaruan bobot nyaris tidak terjadi.

Exploding Gradient

Jika bobot awal terlalu besar (> 1), gradien akan membesar tak terkendali. Hal ini menyebabkan lompatan optimasi yang drastis, menyebabkan nilai Loss menjadi NaN (Not a Number).



Performance Metric

Performance metrics (metrik kinerja) adalah bagian dari setiap machine/deep learning pipeline.

Hal ini merupakan pengukuran secara kuantitatif yang digunakan untuk menilai kemampuan dan efektifitas dari model statistik ataupun machine/deep learning.

Classification Metrics
yang umum digunakan

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

Regression Metrics
yang umum digunakan

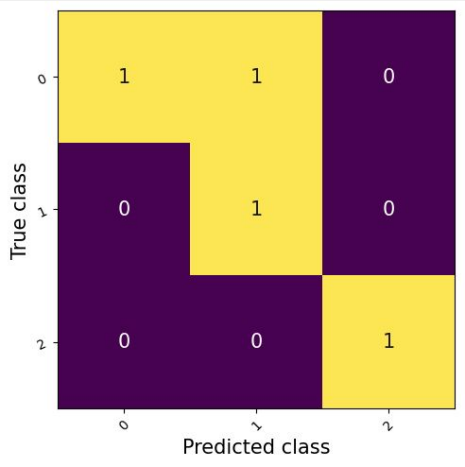
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)
- R-Squared (R2-Score)

Performance Metric

Classification Metrics

Confusion Matrix:

- Representasi matriks yang menampilkan predicted labels dibandingkan true labels untuk mengevaluasi model klasifikasi.
- Matriks ini membantu memvisualisasikan prediksi true positive, true negative, false positive dan false negative.




```
from torchmetrics.classification import
MulticlassConfusionMatrix
func_metrics = MulticlassConfusionMatrix(num_classes=3)
target = torch.tensor([2, 1, 0, 0])

preds_float = torch.tensor([[0.16, 0.26, 0.58],
                            [0.22, 0.61, 0.17],
                            [0.71, 0.09, 0.20],
                            [0.05, 0.82, 0.13]])
predic_result_float = func_metrics(preds_float, target)
fig_, ax_ = func_metrics.plot(predic_result_float)
```

Performance Metric

Classification Metrics

Accuracy:

- Metrics ini dapat mengukur proporsi prediksi yang benar terhadap total prediksi yang dibuat oleh model.
- Hal ini biasanya digunakan dalam masalah klasifikasi tetapi bisa menyesatkan jika dihadapkan pada imbalanced datasets.

$$\text{Accuracy} = \frac{1}{N}\sum_{i}^{N} 1(y_i = \hat{y}_i)$$

Precision:

- Metrics klasifikasi yang mengukur kemampuan model untuk mengidentifikasi dengan benar (SEBERAPA TEPAT?) kejadian positif (*true positives*) di antara semua kejadian positif yang diprediksi (*true positives + false positives*).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall:

- Metrics klasifikasi yang mengukur kemampuan model untuk mengidentifikasi (SEBERAPA MAMPU MENEMUKAN?) kejadian positif (true positives) di antara semua kejadian positif aktual (true positives + false negatives).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Performance Metric

Classification Metrics - Accuracy, Precision, Recall

```
from torchmetrics.classification import MulticlassAccuracy
from torchmetrics.classification import MulticlassPrecision
from torchmetrics.classification import MulticlassRecall

target = torch.tensor([2, 1, 0, 0, 1, 2, 0])
preds_float = torch.tensor([[0.16, 0.26, 0.58],
                            [0.22, 0.61, 0.17],
                            [0.71, 0.09, 0.20],
                            [0.05, 0.82, 0.13],
                            [0.25, 0.12, 0.63],
                            [0.15, 0.32, 0.53],
                            [0.75, 0.05, 0.20]])
```



```
acc_metrics_single = MulticlassAccuracy(num_classes=3, average=None)
acc_metrics_avg = MulticlassAccuracy(num_classes=3)
acc_result_single = acc_metrics_single(preds_float, target)
acc_result_avg = acc_metrics_avg(preds_float, target)
print("Acc pada setiap class = ", acc_result_single)
print("Acc AVG = ", acc_result_avg)

prec_metrics_single = MulticlassPrecision(num_classes=3,
average=None)
prec_metrics_avg = MulticlassPrecision(num_classes=3)
prec_result_single = prec_metrics_single(preds_float, target)
prec_result_avg = prec_metrics_avg(preds_float, target)
print("Prec pada setiap class = ", prec_result_single)
print("Prec AVG = ", prec_result_avg)

rec_metrics_single = MulticlassRecall(num_classes=3, average=None)
rec_metrics_avg = MulticlassRecall(num_classes=3)
rec_result_single = rec_metrics_single(preds_float, target)
rec_result_avg = rec_metrics_avg(preds_float, target)
print("Rec pada setiap class = ", rec_result_single)
print("Rec AVG = ", rec_result_avg)
```



Berapa nilai yang dari Acc, Prec dan Rec yang dihasilkan?

```
Acc pada setiap class =  tensor([0.6667, 0.5000, 1.0000])
Acc AVG =  tensor(0.7222)
Prec pada setiap class =  tensor([1.0000, 0.5000, 0.6667])
Prec AVG =  tensor(0.7222)
Rec pada setiap class =  tensor([0.6667, 0.5000, 1.0000])
Rec AVG =  tensor(0.7222)
```

Performance Metric

Classification Metrics

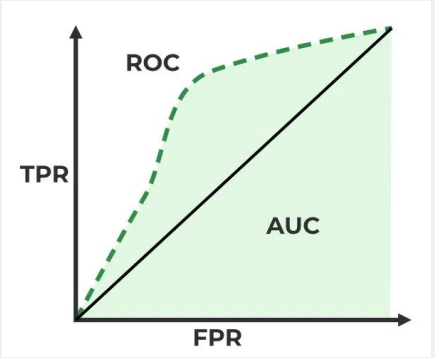
F1-Score:

- Metrics yang digunakan dalam domain klasifikasi yang menyeimbangkan precision dan recall.
- Nilainya berkisar antara 0 dan 1, dengan nilai yang lebih tinggi menunjukkan performa model yang lebih baik.

$$F_1 = 2\frac{\text{precision} * \text{recall}}{(\text{precision}) + \text{recall}}$$

ROC-AUC:

- The area under (AU) the receiver operating characteristic (ROC) curve yang memetakan tingkat true positive model terhadap tingkat false positive untuk nilai ambang batas yang berbeda.
- Metric ini dapat membantu menentukan kemampuan model untuk membedakan antara kelas positif dan negatif.
- Skor AUC-ROC yang lebih tinggi menunjukkan performa model yang lebih baik.



$$\text{TPR} = \frac{TP}{\text{Actual Positive}} = \frac{TP}{TP + FN}$$

$$\text{FPR} = \frac{FP}{\text{Actual Negative}} = \frac{FP}{TN + FP}$$

Performance Metric

Regression Metrics

Mean Squared Error (MSE):

- Regression metric yang menghitung rata-rata selisih kuadrat antara predicted values dan actual values.
- Metric ini membantu dalam mengukur kinerja model pada continuous data dengan memberikan sanksi terhadap kesalahan yang lebih besar (sensitif terhadap outlier data).

$$\text{MSE} = \frac{1}{N}\sum_{i}^{N}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE):

- Akar kuadrat dari MSE, menunjukkan standar deviasi dari kesalahan dan memberikan nilai kesalahan dalam satuan yang sama dengan data.
- Hal ini berguna untuk memahami perbedaan rata-rata antara predicted values dan actual values.

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i}^{N}(y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE):

- MAE yang menghitung rata-rata perbedaan absolut antara predicted values dan actual values.
- Metric ini lebih tahan terhadap outlier dibandingkan MSE.

$$\text{MAE} = \frac{1}{N}\sum_{i}^{N}|y_i - \hat{y}_i|$$

Performance Metric

Regression Metrics

Mean Absolute Percentage Error (MAPE):

- Juga dikenal sebagai mean absolute percentage deviation (MAPD).
- Digunakan untuk mengukur besaran rata-rata kesalahan yang dihasilkan oleh suatu model, atau seberapa jauh rata-rata prediksi meleset.

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{\max(\epsilon, |y_i|)}$$

R-Squared (R2-Score):

- Menjelaskan proporsi varians pada variabel target yang dapat dijelaskan oleh prediktor model.
- R2 berkisar antara 0 dan 1, dengan nilai yang lebih tinggi menunjukkan kecocokan model yang lebih baik.

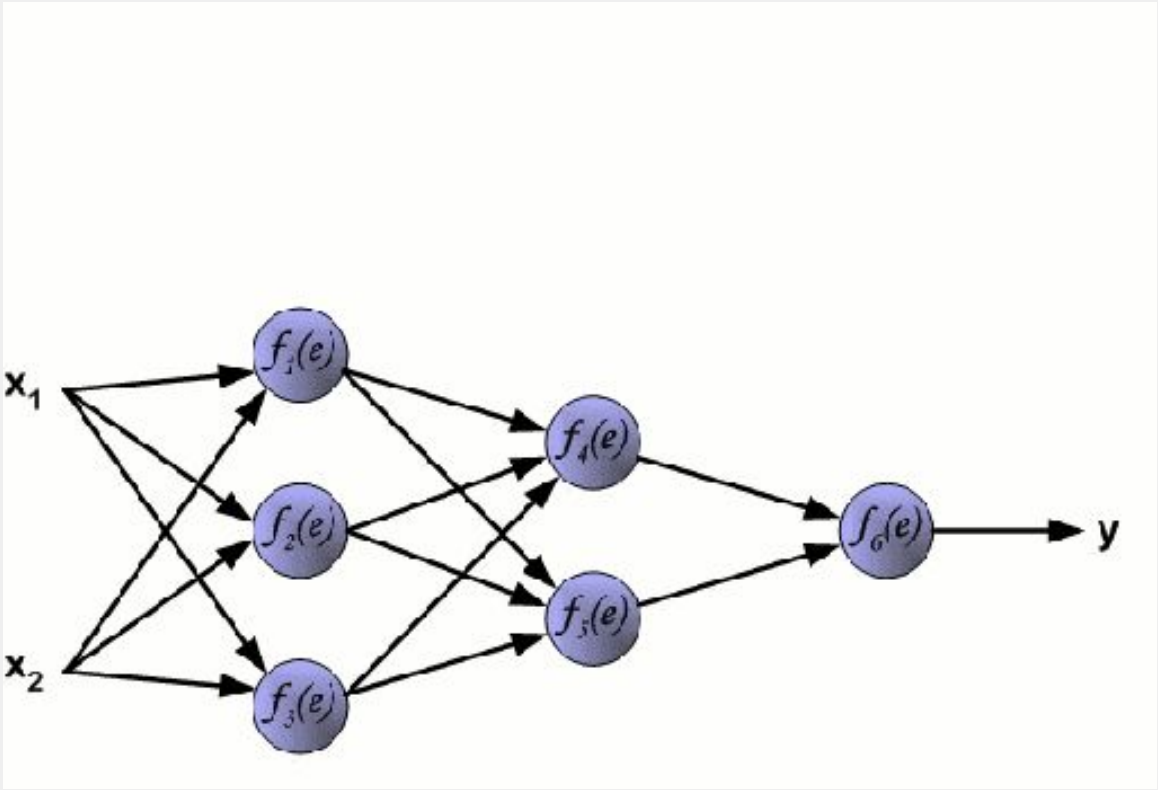
$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} = \sum_i (y_i - f(x_i))^2$$

$$SS_{tot} = \sum_i (y_i - \bar{y})^2$$

$$\bar{y} = \frac{1}{N}\sum_i^n y_i$$

Multi Layer Perceptron (Review)



- Goal: set bobot untuk minimasi error atau metrik tertentu
- Propagasi maju: hitung output y setiap instance x
- Propagasi mundur: hitung error δ setiap neuron
- Proses belajar: Update setiap bobot

Multi Layer Perceptron (Review)

Backpropogation: Phase

Tahap pelatihan:

- Propagasi maju (*Feed Forward*):
  - Masukan dihitung maju mulai dari input layer hingga output layer menggunakan fungsi aktivasi yang dipilih.
- Propagasi mundur (*Backward Phase*):
  - Selisih antara keluaran jaringan dengan nilai target merupakan error yang terjadi. Error tsb dipropagasi mundur.
- Perubahan bobot:
  - Modifikasi bobot untuk menurunkan error.

Ketiga tahap tersebut diulang terus hingga kondisi berhenti terpenuhi.

Multi Layer Perceptron (Review)

Algoritma Backpropogation – Stochastic Gradient Descent

- Menginisialisasi semua bobot dengan angka random yang sangat kecil (mendekati nol). Lalu lakukan hal berikut hingga memenuhi kondisi perberhentian
- Untuk setiap training example, Do (*Catatan untuk kondisi penggunaan sigmoid act func*)
  1. Masukkan training example ke Network dan hitung output dari Network
  2. Untuk setiap output unit *k*

$$\delta_k = o_k(1 - o_k)(t_k - o_k)$$

  3. Untuk setiap hidden unit *h*

$$\delta_h = o_h(1 - o_h) \sum_{k \in output} w_{h,k}\delta_k$$

  4. Update setiap bobot network $w_{i,j}$

$$w_{i,j} \leftarrow w_{i,j} + \Delta w_{i,j} \qquad \Delta w_{i,j} = \eta \delta_j x_{i,j}$$

Multi Layer Perceptron (Review)

Latihan: Simulasikan ANN Learning Sederhana

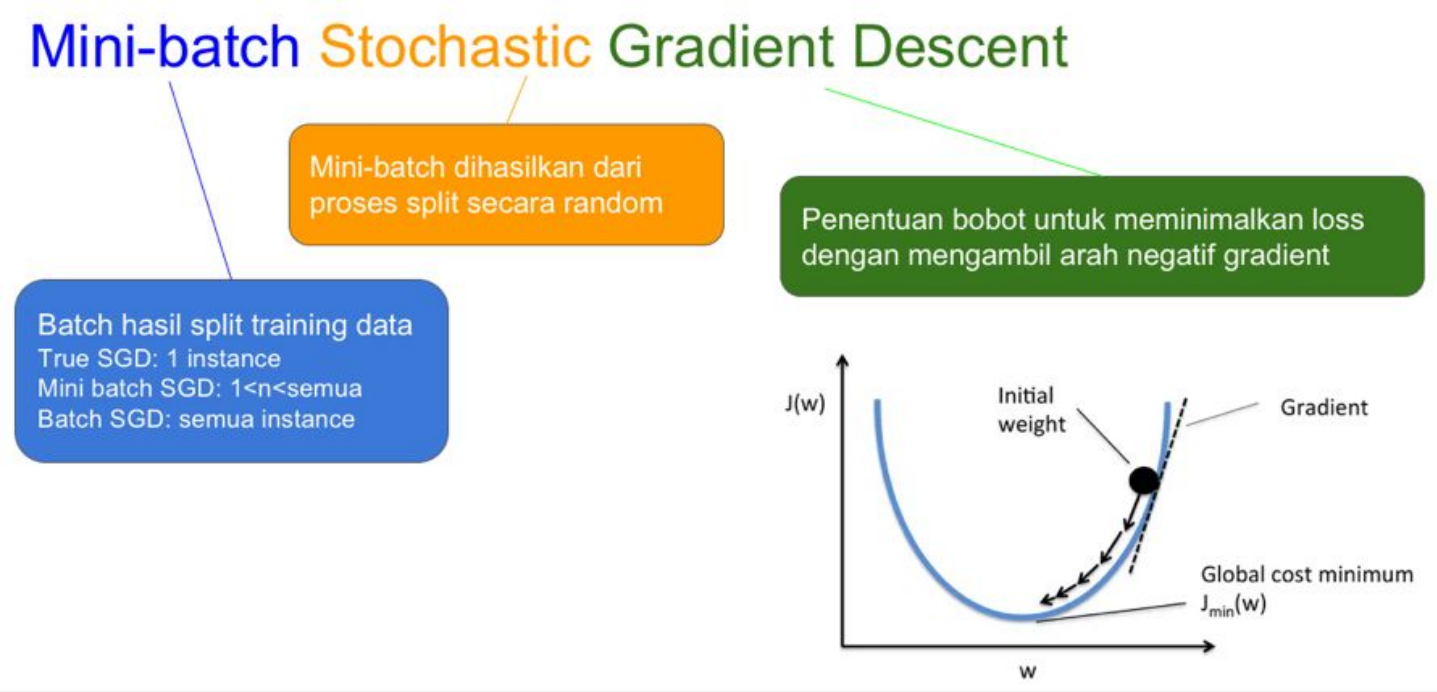
- Diberikan suatu struktur Artificial Neural Net dengan 3 variabel input, 1 hidden layer dengan 2 neuron, dan 2 target/output. Penggunaan sigmoid activation function pada hidden dan output layer.
- Apabila diketahui bobot yang dilibatkan pada struktur ANN tersebut seperti yang terlihat pada tabel, dan diproses suatu data dengan nilai atribut (X_1=1 ; X_2=-0,5 ; X_3=0,5) serta target/label (Y_1=0,15 ; Y_2=0,75).
- Serta jika dilibatkan bias=1 pada input layer dan hidden layer, serta learning rate sebesar 0,1, maka tentukanlah:
  a. Nilai output dari O_1
  b. Delta error pada neuron ke-2 di hidden layer
  c. Update bobot yang menghubungkan X_3 dan H_2

W	H_1	H_2
X_0	-0,2	-0,15
X_1	0,1	0,2
X_2	0,2	-0,15
X_3	0,5	-0,25

W	Y_1	Y_2
H_0	-0,15	0,1
H_1	0,1	0,2
H_2	0,15	-0,1

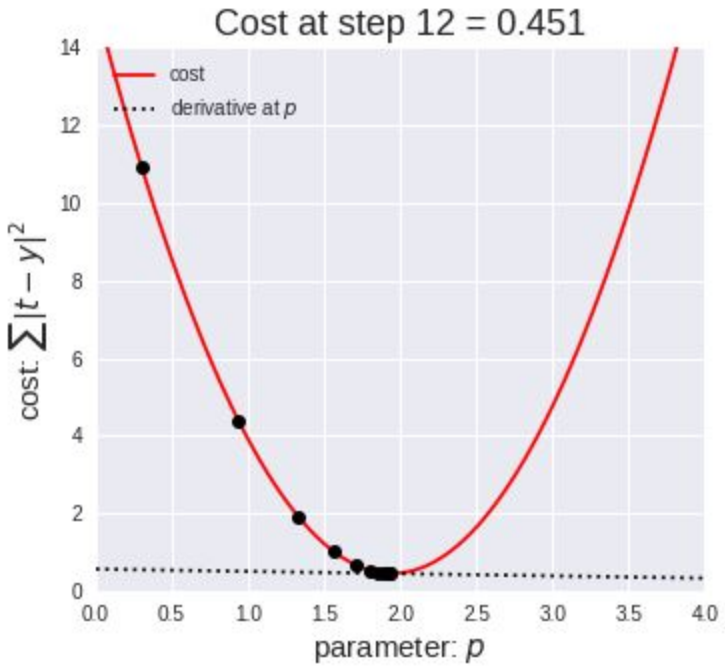
Multi Layer Perceptron (Review)

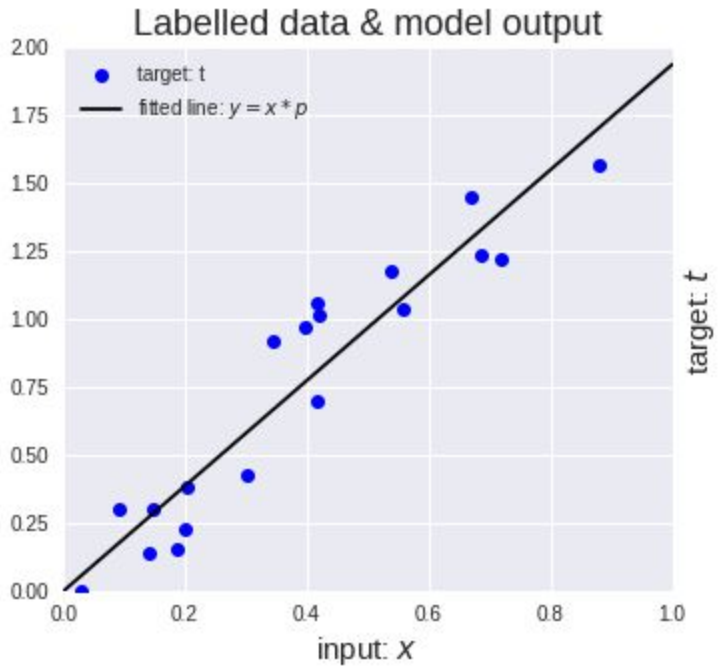
Mini-Batch in Artificial Neural Network



Multi Layer Perceptron (Review)

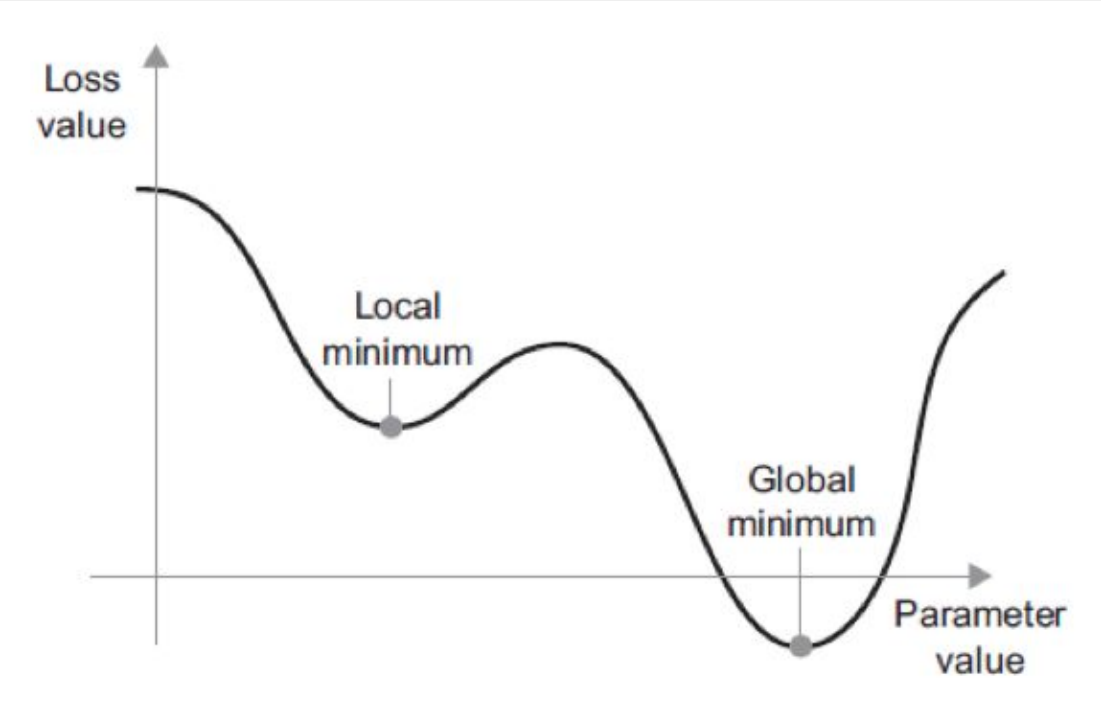
Pencarian Loss Minimum





Multi Layer Perceptron (Review)

Pencarian Bobot Terbaik: Global vs Local Minimum



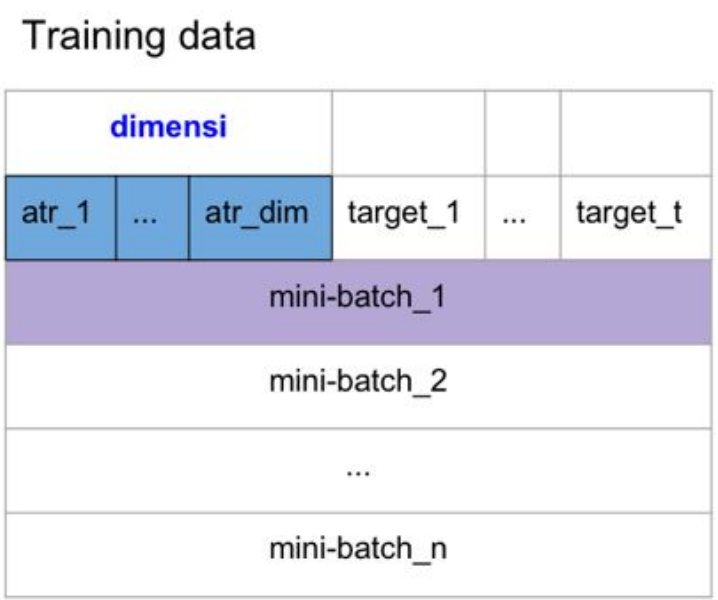
Multi Layer Perceptron (Review)

ANN Learning: Epoch, Step, & Batch Size

Epoch
Iterasi pada backpropagation untuk semua training data atau semua mini-batch.

Batch size
parameter ukuran mini-batch (hasil split training data).
Jika batch size = 1, proses incremental/true SGD
Jika batch size = |training data|, proses batch SGD
Jika 1<batch size<|training data|, proses mini-batch SGD

Step
Pemrosesan satu mini batch diakhiri dengan update bobot

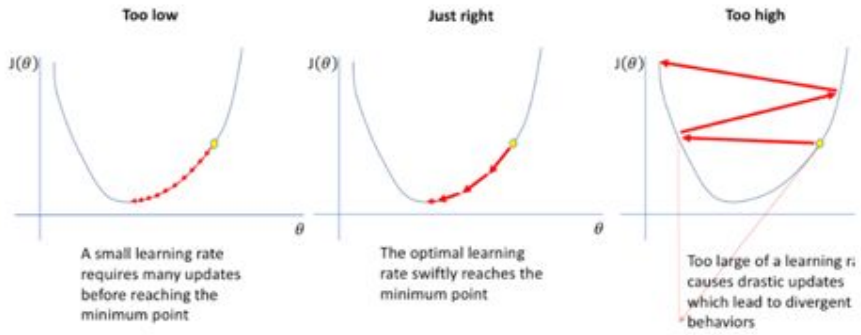


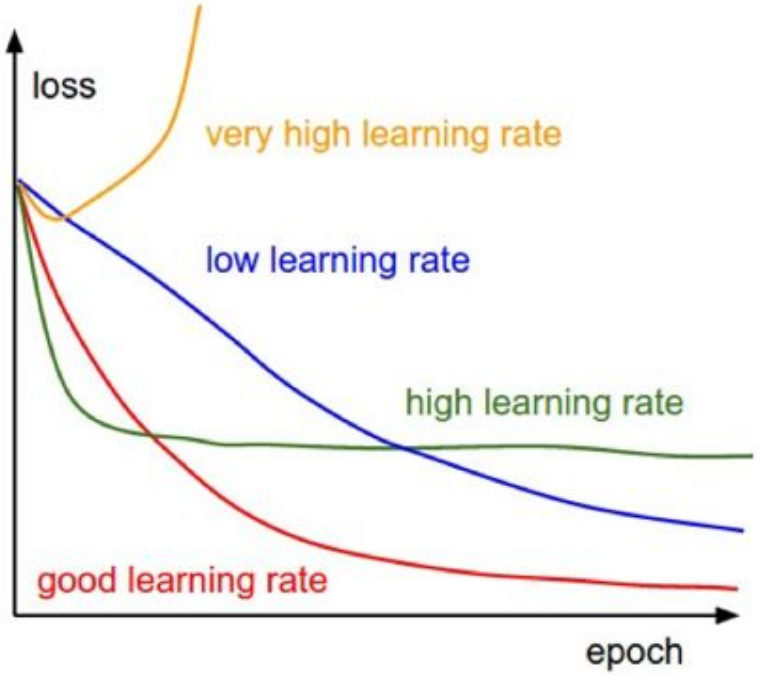
Multi Layer Perceptron (Review)

ANN Learning: Learning Rate

Parameter η untuk update bobot

Jika nilai η terlalu kecil maka konvergensi akan lambat dan dapat terjebak di local minimal. Jika terlalu besar dapat terjadi overshoot bahkan divergen.





Terima Kasih.

Pembahasan Berikutnya: Training dari DL Model

- *Scheduler*
- *Early Stopping*
- *Normalization*
- *Regularization*

